জোড়ায় ল্যাব: সবচেয়ে সাধারণ ভুল এবং কাজের পদ্ধতি

পাঠ 16 — 1IOT, SST-ICT-01 — C·ARENA প্ল্যাটফর্মে প্রথম চ্যালেঞ্জ

09/11/2026

1 আজ আমরা কী করেছি

আজ আমরা তিন ঘণ্টা **নিজে নিজে** (**in autonomia**), জোড়ায় জোড়ায়, **C·ARENA** প্ল্যাটফর্মে কাজ করেছি: একজন **পাইলট** (**pilota**) কোড লেখে, একজন **নেভিগেটর** (**navigatore**) পরিকল্পনা তৈরি করে এবং তাকে পথ দেখায়। অনুশীলনগুলো ছিল পাঠ 15-এর মতোই: **উল্টো গণনা** (**conto alla rovescia**, 1B) এবং গণনা ও যোগফলসহ **100 পর্যন্ত গুণিতক** (**multipli fino a 100**, 2B)। ধাপে ধাপে ব্যাখ্যা করা সমাধান আছে **A15 হ্যান্ডআউটে**।

প্ল্যাটফর্মটি প্রোগ্রাম **কম্পাইল** করে এবং কিছু **টেস্ট** (**test**) দিয়ে পরীক্ষা করে: ইনপুট হিসেবে একটি সংখ্যা দেয় এবং প্রোগ্রাম যা প্রিন্ট করে তা **প্রত্যাশিত আউটপুট** (**uscita attesa**)-এর সাথে মেলায়। আউটপুট **ঠিক যেমন হওয়া উচিত** তেমন হলে, লেখাগুলোসহ, তবেই টেস্ট সবুজ হয়।

ফলাফল এমন হয়েছে: প্রায় সব প্রোগ্রাম সমাধানের **কাছাকাছি** ছিল, কিন্তু খুব কম প্রোগ্রামের সব টেস্ট সবুজ হয়েছে। ভুলগুলো প্রায় সবসময় **একই রকম** ছিল, এবং প্রায় সবসময় যুক্তির চেয়ে বেশি **পদ্ধতির** ভুল। এই হ্যান্ডআউটে তুমি সেগুলো একে একে পাবে, সংশোধন আর মনে রাখার নিয়মসহ। শেষে আছে পরের বারের **পদ্ধতি** এবং টেস্ট চালানোর আগে ব্যবহারের জন্য একটি **চেকলিস্ট**।

2 যে টেস্ট পাস করে না, তা কীভাবে পড়তে হয়

কোনো টেস্ট পাস না করলে প্ল্যাটফর্ম দুটি জিনিস দেখায়: **প্রত্যাশিত আউটপুট** আর **প্রোগ্রামের আউটপুট** (**uscita del programma**)। কাজ হলো এগুলো **লাইন ধরে ধরে মেলানো**, পার্থক্য খোঁজার খেলার মতো।

উদাহরণ

টেস্ট «parto da 5», ইনপুট 5। প্রত্যাশিত আউটপুট:

```
5 4 3 2 1
  Via!
```

প্রোগ্রামের আউটপুট:

```
5 4 3 2 1
  via!
```

একমাত্র পার্থক্য হলো **ছোট হাতের v**। প্রোগ্রামটি 99% ঠিক, কিন্তু টেস্ট লাল।

প্রশ্নের বার্তাগুলো (`Inserisci un numero…`) যেকোনোভাবে লেখা যায়: টেস্ট এগুলো দেখে না। কিন্তু **ফলাফলের লেখাগুলো** অবশ্যই কাজের উদাহরণের সাথে **হুবহু এক** হতে হবে: একই বড় হাতের অক্ষর, একই কোলন (:), একই স্পেস।

> **সতর্কতা**
>
> **0টি টেস্ট পাস করে জমা দেওয়া কোনো কাজে আসে না।** এতে কোনো পয়েন্ট পাওয়া যায় না, আর সংশোধনের সুযোগও চলে যায়। টেস্ট সবুজ হলে জমা দাও, অথবা যখন যতটা সম্ভব সব করেছ এবং বুঝেছ কী বাকি আছে।

3 কম্পাইলের ভুল: বার্তাটি পড়ো

আজকের সবচেয়ে ব্যয়বহুল ভুল ছিল **কম্পাইলারের বার্তা না পড়া**। বার্তাটি বলে **কোন লাইনে** এবং **কী ঠিক নেই**: এটি পড়তে হবে, উপেক্ষা করা যাবে না।

```
#include <stdio.h>

int main()
{
  int n,x
  do {
```

এখানে `int n,x`-এর পরে সেমিকোলন (;) নেই, **লাইন 5**-এ। প্ল্যাটফর্মের কম্পাইলার এটি টের পায় শুধু পরের লাইন পড়ার সময়, তাই ভুলটি **লাইন 6** দেখাতে পারে: এটা স্বাভাবিক। নিয়ম: **দেখানো লাইন এবং তার আগের লাইন দেখো**।

বার্তাটি ইংরেজিতে, কিন্তু কয়েকটি শব্দ চিনলেই যথেষ্ট:

বার্তায় আছে…	এর মানে কী এবং কোথায় দেখতে হবে
`';'`	একটি সেমিকোলন নেই: দেখানো লাইনের শেষ এবং **তার আগের লাইনের** শেষ দেখো
`undeclared`	তুমি এমন একটি ভেরিয়েবল ব্যবহার করছ যা ওই জায়গায় **নেই**: তুমি এটি ঘোষণা করোনি, অথবা `for`-এর ভেতরে ঘোষণা করে বাইরে ব্যবহার করছ
`'}'`	একটি **বন্ধ কার্লি ব্র্যাকেট** নেই: { আর } গুনে দেখো
`expression`	একটি অংশ নেই: যেমন অর্ধেক লেখা একটি শর্ত, `mese < 1 || > 12` এর মতো

> **সতর্কতা**
>
> **একই কোড আবার চালালে কিছুই বদলায় না।** প্রোগ্রাম না বদলে «Lancia i test» চাপলে ফলাফল একই হয়: একই ভুল, একই লাইন। আগে **পড়ো**, তারপর **ঠিক করো**, তারপর **আবার চালাও**।

3.1 কার্লি ব্র্যাকেট এবং ইনডেন্টেশন

প্রতিটি {-এর নিজস্ব } আছে। কোড যদি **ইনডেন্ট করা** (**indentato**) থাকে (প্রতিটি ব্লকের ভেতরে ডানদিকে সরানো), তাহলে কোন ব্র্যাকেট নেই তা সঙ্গে সঙ্গে দেখা যায়। সব যদি বাম মার্জিনে লেগে থাকে, তাহলে খুঁজে পাওয়া প্রায় অসম্ভব।

```
for (int i = n; i <= 100; i += n)
{
    printf("%d ", i);
    somma += i;
    conteggio++;
}
printf("\nMultipli fino a 100: %d\n", conteggio);
```

ভেরিয়েবল i-ও এভাবে ভালো বোঝা যায়: এটি **for-এর ভেতরে** ঘোষিত, তাই এটি শুধু **তার কার্লি ব্র্যাকেটের মধ্যে** থাকে। বন্ধ ব্র্যাকেটের পরে এটি আর নেই: এজন্য গণনা আলাদা একটি ভেরিয়েবলে রাখতে হয়, যা লুপের আগে ঘোষণা করা হয়।

4 সংখ্যা পড়ার সময় ভুল

4.1 উদ্ধৃতিচিহ্নের ভেতরে &

```
scanf("%d, &n");
```

প্রোগ্রাম কম্পাইল হয়, কিন্তু **কাজ করে না**: &n **উদ্ধৃতিচিহ্নের ভেতরে** চলে গেছে, তাই scanf জানে না সংখ্যাটি কোথায় রাখবে। প্রায়ই প্রোগ্রাম আটকে যায় বা কিছুই প্রিন্ট করে না। সঠিক রূপ: উদ্ধৃতিচিহ্ন %d-এর **ঠিক পরেই** বন্ধ হয়, তারপর কমা, তারপর &n।

```
scanf("%d", &n);
```

4.2 do while-এর শর্ত

do while সংখ্যাটি **ভুল** থাকা পর্যন্ত **প্রশ্নটি আবার করে**। তাই শর্তটি ভুল সংখ্যাকে বর্ণনা করে, **কাজের নির্দেশের একই সীমা** দিয়ে।

নির্দেশ	ভুল	সঠিক
1 থেকে 20	`while (n < 1 || n < 20);`	`while (n < 1 || n > 20);`
2 থেকে 9	`while (n < 2 || n > 20);`	`while (n < 2 || n > 9);`

প্রথম ক্ষেত্রে প্রোগ্রাম প্রায় সব ভালো সংখ্যা বাতিল করে। দ্বিতীয় ক্ষেত্রে এটি 10-ও গ্রহণ করে, যা নির্দেশে নিষিদ্ধ: এজন্যই «numeri sbagliati prima» টেস্টটি আছে, যা প্রথমে সীমার বাইরের সংখ্যা দেয় এবং দেখে প্রোগ্রাম সেগুলো **আবার চায়** কিনা।

5 লুপের ভুল

লুপ লেখার আগে **তিনটি প্রশ্নের** উত্তর দিতে হয়: **কোথা থেকে শুরু করি? কখন থামি? প্রতিবার কী বদলায়?** লুপের প্রায় সব ভুল আসে একটি ভুল বা না-দেওয়া উত্তর থেকে।

5.1 যে লুপ কখনো শুরু হয় না

```
for (int a = numero; a >= numero * 10; a--)
{
    printf("%d ", a);
}
```

numero যদি 5 হয়, শর্তটি হয় 5 >= 50: **প্রথম ঘোরাতেই মিথ্যা**। লুপটি একবারও ঘোরে না, আর পর্দায় শুধু তার পরের জিনিসটি দেখা যায়। উল্টো গণনার তিনটি উত্তর হলো: **সংখ্যাটি** থেকে শুরু করি, **অন্তত 1 থাকা পর্যন্ত** চালিয়ে যাই, প্রতিবার **এক করে কমাই**।

```
for (int a = numero; a >= 1; a--)
{
    printf("%d ", a);
}
```

5.2 যে লুপ শেষ হয় না

প্ল্যাটফর্ম যদি বলে প্রোগ্রাম **শেষ হয় না**, তাহলে শর্তটি **সবসময় সত্য**: ভেরিয়েবলটি বদলায় না, অথবা ভুল দিকে বদলায়। উদাহরণ: n থেকে শুরু করি, 1 পর্যন্ত নামতে চাই, কিন্তু i---এর বদলে i++ লিখি: i বাড়তে থাকে, আর i >= 1 চিরকাল সত্য থাকে।

5.3 শর্ত মানের উপর, ঘোরার সংখ্যার উপর নয়

100 পর্যন্ত গুণিতকে লুপ থামে যখন **গুণিতকটি** 100 পার হয়, নির্দিষ্ট সংখ্যক ঘোরার পরে নয়।

```
for (int a = 2; a <= 9; a++)
{
    printf("%d ", n * a);
}
```

7 দিলে এই লুপ 14 থেকে 63 প্রিন্ট করে: **7 নেই** (1-এর বদলে 2 থেকে শুরু হয়) এবং **খুব আগে থামে** (98-এর বদলে 63)। সঠিক লুপ সংখ্যাটি থেকেই শুরু হয়, n করে বাড়ে এবং 100 পার হলে থামে:

```c
for (int i = n; i <= 100; i += n)
{
    printf("%d ", i);
}
```

ধাপের দিকেও খেয়াল রাখো: লেখা হয় `i += n`, `i = i += n` নয়।

6 প্রিন্ট করার ভুল

এগুলো সবচেয়ে হতাশাজনক ভুল: প্রোগ্রাম **ঠিকঠাক হিসাব করে**, কিন্তু টেস্ট লাল কারণ এটি **ভিন্নভাবে প্রিন্ট করে**।

6.1 সংখ্যা জোড়া লাগানো বা প্রতি লাইনে একটি

নির্দেশ (instruction)	পর্দায়	ঠিক আছে?
`printf("%d", i);`	714212835…	না: সংখ্যা জোড়া লাগানো
`printf("%d\n", i);`	প্রতি লাইনে একটি সংখ্যা	না: নির্দেশ এগুলো এক লাইনে চায়
`printf("%d ", i);`	`7 14 21 28 …`	হ্যাঁ

6.2 লেখাগুলো হুবহু এক হতে হবে

প্রোগ্রামে লেখা	টেস্ট যা চায়
`via!`	`Via!`
`multipli fino a 100: 14`	`Multipli fino a 100: 14`
`somma 735`	`Somma: 735`

নিয়মটি সহজ: ফলাফলের লেখাগুলো নির্দেশের **উদাহরণ থেকে কপি করো**, অক্ষরে অক্ষরে, **সঙ্গে সঙ্গে**, printf লেখার সময়েই। শেষে এক টেস্টের পর আরেক টেস্টে এগুলো ঠিক করতে গেলে অনেক সময় নষ্ট হয়।

6.3 যে লাইনটি নেই

প্রায়ই প্রোগ্রামটি ঠিক ছিল **শুধু শেষ লাইনটি ছাড়া**: উল্টো গণনার পরে `Via!` ছিল না, অথবা গণনা আর যোগফলের লাইন ছিল না। প্রত্যাশিত আউটপুটের সাথে লাইন ধরে মেলালে এটি সঙ্গে সঙ্গে দেখা যায়। `Via!` যাবে **নতুন লাইনে** এবং **লুপের পরে**, ভেতরে নয়:

```
for (int i = n; i >= 1; i--)
{
    printf("%d ", i);
}
printf("\nVia!\n");
```

7 गणक एवं योगफल

2B-তে, গুণিতকের পরে আরও দুটি লাইন লাগে: সেগুলো **কয়টি** এবং তাদের **যোগফল**। দুটি ভেরিয়েবল লাগে:

- একটি **গণক** (**contatore**): 0 থেকে শুরু হয় এবং প্রতিবার 1 করে বাড়ে (`conteggio++`);
- একটি **সঞ্চয়ক** (**accumulatore**): 0 থেকে শুরু হয় এবং প্রতিবার সংখ্যাটি যোগ করে (`somma += i`)।

```
int conteggio = 0;
int somma = 0;

for (int i = n; i <= 100; i += n)
{
    printf("%d ", i);
    conteggio++;
    somma += i;
}

printf("\nMultipli fino a 100: %d\n", conteggio);
printf("Somma: %d\n", somma);
```

```
7 14 21 28 35 42 49 56 63 70 77 84 91 98
Multipli fino a 100: 14
Somma: 735
```

সাধারণ ভুল তিনটি:

- গণক আর যোগফল **0 না করা**: `= 0` ছাড়া `int cicli;` একটি এলোমেলো মান থেকে শুরু হয়, আর প্রিন্ট করা গণনা একটি অদ্ভুত সংখ্যা হয় (যেমন `-162758390`);
- **হিসাব করা কিন্তু প্রিন্ট না করা**: ভেরিয়েবলগুলো আছে, কিন্তু লুপের পরে দুটি printf ছাড়া লাইনগুলো দেখা যায় না;
- **লুপের ভেতরে প্রিন্ট করা**: শেষে একবারের বদলে প্রতিবার বের হয়।

8 পরের বারের পদ্ধতি

আজকের প্রায় সব ভুল একটি **পদ্ধতি** দিয়ে এড়ানো যায়। এই সেই পদ্ধতি।

1. **কোডের আগে পরিকল্পনা।** নেভিগেটর তিনটি উত্তর লেখে: কোথা থেকে শুরু, কখন থামি, প্রতিবার কী বদলায়। পরিকল্পনা না থাকা পর্যন্ত পাইলট লেখে না।

2. **একবারে একটি অংশ, আর সঙ্গে সঙ্গে পরীক্ষা।** প্রথমে শুধু do while দিয়ে সংখ্যা চাওয়া: চালাও এবং দেখো। তারপর লুপ: চালাও এবং দেখো। তারপর শেষ লাইন। তারপর গণক আর যোগফল। পুরো প্রোগ্রাম লিখে তারপর একসাথে বিশটি ভুল আবিষ্কার করা নয়।
3. **আগে আগে পরীক্ষা করো।** অসম্পূর্ণ প্রোগ্রামও চালিয়ে দেখা যায় সেটা কী করে। কোনো পরীক্ষা ছাড়া আধা ঘণ্টা বা এক ঘণ্টা কাটানো মানে অন্ধকারে কাজ করা।
4. **একবারে একটি পরিবর্তন।** তিনটি জিনিস বদলালে আর টেস্ট তখনও লাল থাকলে, আমি জানি না তিনটির কোনটি ভুল ছিল।
5. **আবার চালানোর আগে পড়ো।** কম্পাইলের ভুল: লাইন আর বার্তা পড়ি। লাল টেস্ট: প্রত্যাশিত আউটপুট আমার আউটপুটের সাথে লাইন ধরে মেলাই।
6. **লেখাগুলো উদাহরণ থেকে।** সঙ্গে সঙ্গে হুবহু কপি করি।
7. **সাহায্য (aiutini) কাজে লাগে**, কিন্তু 10% পয়েন্ট কেটে নেয়: সত্যিই চেষ্টা করার পরে ব্যবহার করো।

টীকা

পাইলট এবং নেভিগেটর। **পাইলট** লেখে এবং টেস্ট চালায়। **নেভিগেটর** শুধু তাকিয়ে থাকে না: পরিকল্পনা রাখে, লেখাগুলো উদাহরণের সাথে মেলায়, ভুলের বার্তা জোরে পড়ে এবং আউটপুট মেলায়। একই প্রোগ্রাম দুজন দেখলে একজনের চেয়ে অনেক আগে ভুল খুঁজে পায়।

9 চেকলিস্ট: «Lancia i test» চাপার আগে

«Lancia i test» চাপার আগে

- [ ] শেষ পরীক্ষার পর আমি **কিছু বদলেছি**?
- [ ] প্রতিটি নির্দেশ কি **সেমিকোলন** দিয়ে শেষ হয়? প্রতিটি {-এর নিজস্ব } আছে?
- [ ] scanf কি `scanf("%d", &n);` এভাবে লেখা, & উদ্ধৃতিচিহ্নের বাইরে?
- [ ] do while-এর শর্ত কি **নির্দেশের সীমা** ব্যবহার করে?
- [ ] লুপ: **কোথা থেকে শুরু**, **কখন থামে**, **কী বদলায়**? প্রথম আর শেষ ঘোরা হাতে হিসাব করে দেখেছি?
- [ ] সংখ্যার printf-এ %d-এর পরে **স্পেস** আছে?
- [ ] লেখাগুলো কি **উদাহরণের সাথে হুবহু এক**, বড় হাতের অক্ষর আর কোলনসহ?
- [ ] **শেষ লাইনটি** (`Via!`, গণনা, যোগফল) কি লুপের পরে আছে?
- [ ] গণক আর যোগফল কি **0** থেকে শুরু হয়?

10 শব্দকোষ: ক্লাসে ব্যবহৃত ইতালীয় শব্দ

- **pilota / navigatore**: পাইলট (যে লেখে) / নেভিগেটর (যে পথ দেখায়)
- **piano**: পরিকল্পনা

- **Lancia i test**: টেস্ট চালাও
- **Esegui / Consegna**: চালাও / জমা দাও
- **uscita attesa / uscita del programma**: প্রত্যাশিত আউটপুট / প্রোগ্রামের আউটপুট
- **aiutino**: ছোট সাহায্য (ইঙ্গিত)
- **non compila**: কম্পাইল হয় না
- **conto alla rovescia**: উল্টো গণনা
- **Via!**: যাও!
- **multipli fino a 100**: 100 পর্যন্ত গুণিতক
- **conteggio / somma**: গণনা / যোগফল
- **numeri sbagliati prima**: আগে ভুল সংখ্যা
- **da dove parto? quando mi fermo? che cosa cambia?**: কোথা থেকে শুরু? কখন থামি? কী বদলায়?